Date...../...../.....

MySQL Notes for Beginners

1. What is MySQL?

MySQL is a relational database management system (RDBMS) used to store, organize, retrieve and manage data using SQL.

Example: A company can store employee details salaries, customers and sales record in MySQL.

2. Important MySQL terms.

Term	Meaning
Database	Collection of related tables
Table	Data arranged in rows and columns
Row	One record
Column	One attribute or field
Primary Key	Uniquely identifies each row
Foreign Key	Connects related tables
SQL	Language used to work with databases.

CamScanner

Date...../...../.....

3. Create and select a database.

SQL

```
CREATE DATABASE company;
USE company;
SHOW DATABASES;
SHOW TABLES;
```

- CREATE DATABASE creates a database.

USE selects a database.

SHOW DATABASES list databases.

SHOW TABLES lists tables in the selected databases.

4. Create a table.

SQL

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    department VARCHAR(50),
    salary DECIMAL(10,2)
);
```

Date...../...../.....

Common data types:

Data type	Use
INT	Whole numbers
VARCHAR(n)	Text of variable length
CHAR(n)	Fixed-length text
DECIMAL(p,s)	Exact decimal numbers
DATE	Dates
DATETIME	Dates and Time.
BOOLEAN	True/False-style values.

5. Insert data

```
INSERT INTO employees.
(id, name, age, department, salary)
VALUES
(1, 'RAM', 25, 'IT', 50000),
(2, 'SITA', 28, 'HR', 45000),
(3, 'HARI', 30, 'IT', 60000);
```


CamScanner

Date...../...../.....

6. Retrieve data Using SELECT SQL.

```
-- Display all columns and rows
SELECT * FROM employees;

--Display selected columns
SELECT name, salary
FROM employees;

-- Display unique departments
SELECT DISTINCT Department
FROM employee;
```

SELECT retrieves data from table.

7. Filter data using WHERE SQL

```
-- Employees earning more than 50000.
SELECT *
FROM employees
WHERE salary > 50000;
```


CamScanner

Date...../...../.....

```
--Employees in IT
SELECT *
FROM employees
WHERE department = 'IT';

--Employees aged between 25 and 30
SELECT *
FROM employees
WHERE age BETWEEN 25 AND 30;
```

Useful Operators.

Operators	Meaning.
=	equal to
!= or <>	not equal to
>	Greater than
<	less than
>=	Greater than or equal to
AND	Both Conditions must be True.
OR	At least one condition must be Tru
IN	Matches a value in a list
LIKE	Matches a text ~~PositionPatter~~
BETWEEN	Checks a range

Date...../...../.....

8. Search text using LIKE

SQL

```
-- Names starting with R
SELECT *
FROM employees
WHERE name LIKE 'R%';

-- Names ending with a
SELECT *
FROM employees
WHERE name LIKE '%a';

-- Names containing ar
SELECT *
FROM employees
WHERE name LIKE '%ar%';
```

Where,

% matches zero or more characters

_ matches exactly one character.

Date...../...../.....

9. Sort and limit results.

SQL

```
-- Highest salary first
SELECT *
From employees.
ORDER BY salary DESC;

-- Lowest salary First
SELECT *
FROM employees.
ORDER BY salary ASC;

--Show only the top two Salaries
SELECT *
FROM employees
ORDER BY salary. DESC
LIMIT 2;
```

ASC means ascending: DESC means Descending

CamScanner

Date...../...../.....

10. Aggregate functions

Aggregate functions summarize data.

SQL.

```
SELECT COUNT(*) FROM employee;
SELECT SUM(salary) FROM employee;
SELECT AVG(salary) FROM employee;
SELECT MAX(salary) FROM employee;
SELECT MIN(salary) FROM employee;
```

Function	Purpose.
COUNT()	Counts rows or non-NULL values
SUM()	calculates a total
AVG()	Calculates an average.
~~COUNT()~~ MAX()	Finds the maximum
MIN()	Finds the minimum.

11. GROUP BY and HAVING

SQL

```
-- Average salary by department
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```


CamScanner

Date...../...../.....

SQL

-- Departments with average salary above 50000

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY departments.
HAVING AVG (salary) > 50000;
```

Remember: WHERE filters rows before grouping
HAVING filters groups after grouping.

12. Update and delete data.

SQL

```
-- Update a salary
UPDATE employees
SET salary = 55000
WHERE id = 1;

-- Delete one employee
DELETE FROM employees
WHERE id = 3;
```

Be careful: Without a WHERE condition, UPDATE can modify every row and DELETE can delete every row in table.

Date...../...../.....

13. Modify a table.

SQL

```
-- Add a column
ALTER TABLE employee
ADD email VARCHAR(100);

-- change a column's data type
ALTER TABLE employees
MODIFY name VARCHAR(100);

-- Remove a column
ALTER TABLE employees
DROP COLUMN email;
```

14. NULL values

NULL means a value is missing or unknown

SQL

```
-- finding missing email values
SELECT *
FROM employees
WHERE email IS NULL
```

Date...../...../.....

-- Find values that are not missing

```
SELECT *
FROM employees
WHERE email IS NOT NULL;
```

Do not use Email = NULL. Use IS NULL or IS NOT NULL

15. JOIN operations

JOIN combines related data from multiple tables

Suppose we have these tables:

employees

Id	name	department_Id
1	Ram	10
2	Sita	20
3	Hari	10

departments.

department_id	department_name
10	IT
20	HR
30	Finance

CamScanner

Date...../...../.....

INNER JOIN - returns matching records from both tables.

SQL

```
SELECT e.name, d.department_name.
FROM employee e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

LEFT JOIN - returns every row from the left table including unmatched rows.

SQL.

```
SELECT e.name, d.department_name
FROM employee e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

JOIN	Result.
INNER JOIN	Matching rows only
LEFT JOIN	All left-table rows and matches
RIGHT JOIN	All right-table rows and matches
CROSS JOIN	Every combination of rows

CamScanner

Date...../...../.....

16. Subqueries

A subquery is a query inside another query.

SQL.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

This finds employee earning more than the overall average salary.

17. Constraints

Constraints enforce rules on data

- PRIMARY KEY - uniquely identifies each row
- FOREIGN KEY - maintains relationship between tables
- NOT NULL - prevents missing values.
- UNIQUE - prevents duplicate values in a column or combination
- DEFAULT - supplies a default value
- CHECK - enforces a condition on values.

CamScanner

Date...../...../.....

Example:

SQL

```
CREATE TABLE students (
    id . IN PRIMARY KEY,
    name VARCHAR (50) NOT NULL,
    age INT CHECK (age >= 0),
    city VARCHAR(50) DEFAULT 'KATHMANDU'
);
```

18. SQL command categories.

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP	Defines database structure
DML	INSERT, UPDATE, DELETE	Modify Data
DQL	SELECT	Retrieve Data
DCL	GRANT, REVOKE	Manage permissions
TCL	COMMIT, ROLLBACK	Manage transactions

Recommended learning order: SELECT → WHERE - ORDER BY - aggregate functions → GROUP BY - JOINS → subqueries → window functions → CTEs.

CamScanner